Multilevel Approach to Efficient Gradient Calculation in Stochastic Systems

Joohwan Ko[1,4] **Michael Poli**[2,4] **Stefano Massaroli**[3,4] **Jinkyoo Park**[1] **Woochang Kim**[1]
KAIST[1] Stanford University[2] MILA[3] DiffeqML[4]

Abstract

The advancement of neural network-based approaches for parameterizing Stochastic Differential Equations' vector fields has enhanced the modeling of stochastic systems. Yet, the standard pathwise training for these Neural Stochastic Differential Equations (Neural SDEs) often encounters instability, primarily due to the high variance stemming from reliance on a single Brownian process realization for gradient estimation. We explore the integration of naive Monte Carlo (MC) simulations to enhance stability and convergence through more accurate gradient estimation. However, this approach is computationally demanding, with a cost of $O(\epsilon^{-3})$ required to achieve a gradient mean squared accuracy of ϵ^2. We introduce a novel, scalable approach incorporating Multilevel Monte Carlo (MLMC) into Neural SDE training. MLMC, exploiting different resolution levels in numerical approximation schemes, offers comparable results to MC but at reduced computational costs. Our experiments demonstrate that, compared to standard Neural SDEs, our MLMC-integrated approach significantly improves loss convergence, requiring over 30% fewer simulations than the MC method across both synthetic and real-world datasets.

1 Introduction

Stochastic differential equations (SDEs) are a generalization of ordinary differential equations (ODEs) that are widely used to model stochastic dynamical systems in diverse fields, including physics and finance Øksendal and Øksendal [2003], Gardiner et al. [1985], Black and Scholes [1973], Hull and White [1990]. Recent advancements in neural network-based approaches for parameterizing the vector fields of differential equations have enabled the modeling of even more complex systems by incorporating differential equations into their architectures Chen et al. [2018a], Li et al. [2020]. In particular, for SDEs, infinite-dimensional variational auto-encoders (VAEs) and generative adversarial networks (GANs) have been developed and successfully applied to various applications Kidger et al. [2021a], Li et al. [2020].

Training SDEs-injected neural networks(Neural SDEs) has been a critical challenge due to the inherent stochasticity, which impedes models from converging in the right direction, especially in the early stages of training. In these systems, computing the gradient of the underlying stochastic differential equations (SDEs) requires taking into account different realizations of the Brownian process. Despite extensive research on pathwise gradients, the calculation of expectations over different realizations of the Brownian process in SDEs is often overlooked. Most of the works update the model's parameters with the gradient of the loss calculated by a single realized Brownian process Tzen and Raginsky [2019], Li et al. [2020]. However, relying on batch training with numerous iterations without calculating the accurate gradient of the underlying SDEs mostly results in training instability.

To achieve robust training, it is necessary to compute the accurate gradient of SDEs using Monte Carlo (MC) simulations, which has not been well-discussed in previous literature. In this study, we

Preprint. Under review.

demonstrate that even by integrating a naive MC simulation in these systems, stable training can be achieved with a reduction in gradient variance and better convergence due to accurate gradient calculations. Additionally, we propose a method to reduce computational complexity while achieving similar results as the naive MC approach. Multilevel Monte Carlo (MLMC), an efficient way to estimate gradients, can attain an accuracy of ϵ with a computational complexity of $O(\epsilon^{-2})$ under certain conditions Giles [2015], whereas MC simulations require a computational cost of $O(\epsilon^{-3})$. Both synthetic and real-world datasets show that MC integrated Neural SDEs exhibit robustness in training and better convergence compared to the ones without simulations.

2 Background

2.1 Neural SDEs

Neural SDEs are often used to model the dynamics of stochastic systems [Liu et al., 2019]. They are defined as input-injected SDEs in either Itô or Stratonovich forms. It can be formulated as follows [Kidger et al., 2021b]:

$$Z_0 = \zeta_\theta(X), \quad \mathrm{d}Z_t = f_\theta\left(t, X, Z_t\right)\mathrm{d}t + g_\theta\left(t, X, Z_t\right)\mathrm{d}W_t, \quad Y_t = \ell_\theta\left(Z_t\right), \tag{1}$$

where ζ_θ, f_θ, and g_θ are neural networks, and ℓ_θ is affine.

The training process for Neural SDEs can be generalized by minimizing the expected objective, $\mathcal{L}(Y)$, over different realizations of the Brownian process W_t. It can be achieved by solving a nonlinear program while optimizing parameters for the system, θ. We can write down the nonlinear program constrained to the Neural SDE as equation 2.

$$\begin{aligned} \min_\theta \quad & \mathbb{E}_x\left[\mathbb{E}_W\left[\mathcal{L}(Y)\right]\right] \\ \text{s.t.} \quad & \mathrm{d}Z_t = f_\theta\left(t, X, Z_t\right)\mathrm{d}t + g_\theta\left(t, X, Z_t\right)\mathrm{d}W_t \\ & Z_0 = \zeta_\theta(X), \quad Y_t = \ell_\theta\left(Z_t\right) \end{aligned} \tag{2}$$

We can solve equation 2 via stochastic gradient descent, where we need to calculate $\frac{\mathrm{d}}{\mathrm{d}\theta}\mathbb{E}_W\left[\mathcal{L}(Y)\right]$ [Bottou, 2010] and the SGD iteration has assumptions of $\mathcal{L}$ being Lipschitz such that $\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right] = \frac{\mathrm{d}}{\mathrm{d}\theta}\mathbb{E}_W\left[\mathcal{L}(Y)\right]$. Therefore the exact gradient for the Neural SDE can be estimated by calculating the expectation of pathwise gradients $\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)$ for each of the simulated independent Brownian process.

Latent SDEs. As a prominent instance of Neural SDEs, Latent SDEs[Li et al., 2020] can be characterized as infinite-dimensional VAEs with prior and approximated posterior dictated by two SDEs: $\mathrm{d}\tilde{Z}_t = h(t, \tilde{Z}_t)\mathrm{d}t + g(t, \tilde{Z}_t)\mathrm{d}W_t$ and $\mathrm{d}Z_t = f(t, Z_t)\mathrm{d}t + g(t, Z_t)\mathrm{d}W_t$. The evidence lower bound (ELBO) formula for observing $x_1, x_2, \ldots, x_N$ is given as:

$$\log(p(x_1, x_2, ..., x_N|\theta) \geq \mathbb{E}_{Z_t}\left[\sum_{i=1}^{N}\log p\left(x_{t_i}|z_{t_i}\right) - \int_0^T \frac{1}{2}\left|u\left(t, z_t\right)\right|^2 \mathrm{d}t\right],$$

where $u : \mathbb{R}^d \times [0, T] \to \mathbb{R}^m$ satisfies $g(t, z)u(t, z) = f(t, z) - h(t, z)$, and the expectation is based on the defined approximate posterior process. Latent SDEs will serve as our benchmark model in both synthetic and real-world data experimentation.

2.2 Pathwise Gradients Calculation for SDEs

There is an extensive amount of work on pathwise sensitivities analysis. The forward pathwise approach is an intuitive pathwise gradient calculation method for SDEs [Gobet and Munos, 2005]. However, it scales poorly in computational cost with the number of parameters or state dimensions. Giles and Glasserman [2006] proposed a discrete version of the backward adjoint sensitivity method, which calculates the same value with the forward implementation but with computational savings. The stochastic adjoint method [Li et al., 2020] resolved the memory burden by generalizing the adjoint method to stochastic dynamics defined by SDEs to recover the value of the state z_t.

2.3 Multilevel Monte Carlo

Giles [2008b] proposed MLMC that reduces the order of complexity of MC simulations. When we want to estimate $\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_L(Y)\right]$, there is a sequence $\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_0(Y), \ldots, \frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_{L-1}(Y)$, with increasing accuracy and cost. Then, we can use equation 3 as an unbiased estimator for $\frac{\mathrm{d}}{\mathrm{d}\theta}\mathbb{E}_W\left[\mathcal{L}_L(Y)\right]$.

$$\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_L(Y)\right] = \mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_0(Y)\right] + \sum_{l=1}^{L}\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_l(Y) - \frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_{l-1}(Y)\right] \tag{3}$$

Theorem 2.1 (Complexity Theorem) *For a real-valued random variable $\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)$, we let $\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y})$ be the corresponding approximation using the discretization at level l, i.e. with 2^l steps of width $h_l = 2^{-l}T$.*

If there exists independent estimators $\widehat{U}_l$ of computational complexity C_l based on M_l samples and there are positive constants $\alpha \geq \frac{1}{2}\min(1,\beta), \beta, c_1, c_2, c_3$, such that

$$\begin{aligned}
&A1 : \mathbb{E}\left[\widehat{U}_l\right] = \begin{cases} \mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_0(\widehat{Y})\right] & \text{if } l = 0 \\ \mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_{l-1}(\widehat{Y})\right] & \text{if } l > 0 \end{cases} \\
&A2 : \left|\mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]\right| \leq c_1 h_l^{\alpha} \\
&A3 : \mathbb{V}\left[\widehat{U}_l\right] \leq c_2 h_l^{\beta} M_l^{-1} \\
&A4 : C_l \leq c_3 M_l h_l^{-1}
\end{aligned} \tag{4}$$

Then there is a constant c_4 such that for any $\epsilon < e^{-1}$, there are values for L and $(M_l)_{l=0,\ldots,L}$ resulting in a multilevel estimator $\widehat{U} = \sum_{l=0}^{L}\widehat{U}_l$ with a mean-square-error $MSE = \mathbb{E}\left[(\widehat{U} - \mathbb{E}(\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)))^2\right] < \epsilon^2$ with a complexity C bounded by

$$C \leq \begin{cases} c_4\epsilon^{-2} & \text{if } \beta > 1 \\ c_4\epsilon^{-2}(\log\epsilon)^2 & \text{if } \beta = 1 \\ c_4\epsilon^{-2-(1-\beta)/\alpha} & \text{if } 0 < \beta < 1 \end{cases} \tag{5}$$

The theorem 2.1 [Burgos, 2014] shows its advantages in the convergence speed of V_l, where $V_l = M_l\mathbb{V}[\widehat{U}_l]$, and guarantees improved computational complexity compared to the standard Monte Carlo for stochastic simulation, which we will discuss in detail in the next section.

3 Monte Carlo-Neural SDEs

In this section, we explore the integration of Monte Carlo (MC) simulations into the training of Neural SDEs to improve the accuracy of gradient calculations and stabilize the training process.

3.1 Monte Carlo Integration for Gradient Estimation in Neural SDEs

We first detail the process of integrating MC into Neural SDEs while employing discretization schemes for SDE solutions. Upon deriving the initial SDE value as per equation 1, we produce M independent realizations of the Brownian process $W_t^{(i)}$ for $i = 1, \ldots, M$. Each realization $W_t^{(i)}$ is then used to simulate the Neural SDEs to calculate $Z_t^{(i)}$ using a numerical method, like the Euler-Maruyama scheme:

$$Z_{t+1}^{(i)} = Z_t^{(i)} + f_\theta(t, X, Z_t^{(i)})h + g_\theta(t, X, Z_t^{(i)})\Delta W_t^{(i)},$$

where $\Delta W_t^{(i)} = W_{t+1}^{(i)} - W_t^{(i)}$. With $Y_t = \ell_\theta\left(Z_t\right)$ and $\mathcal{L}$ representing the Lipschitz loss function, we aim to minimize the scalar loss, $\mathcal{L}(Y)$. We use numerical solvers for SDE computation, which

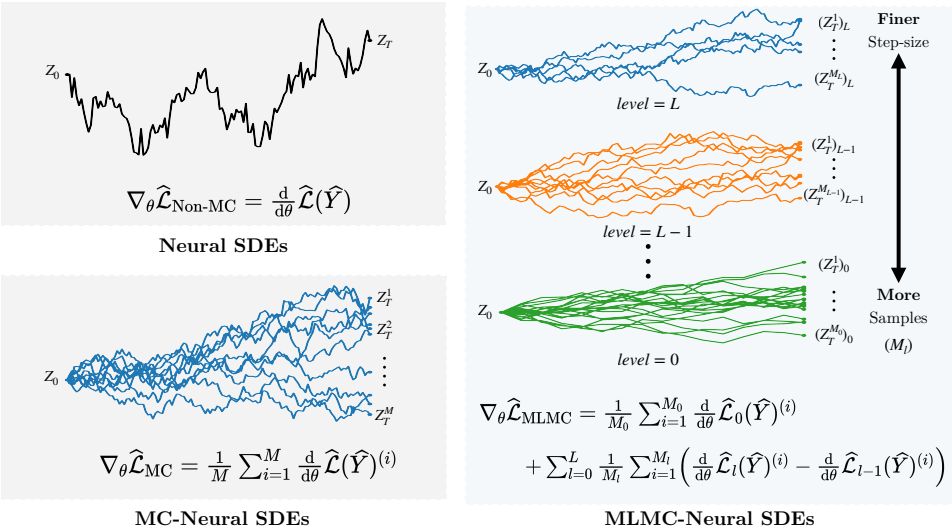

Figure 1: A visual representation of standard Neural SDEs, Monte Carlo (MC)-Neural SDEs, and Multilevel Monte Carlo (MLMC)-Neural SDEs.

rely on a discretization scheme for SDE solution approximation. This allows us to denote $\nabla_\theta \mathcal{L}$ as $\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]$ and represent the discretized $\nabla_\theta \mathcal{L}$ approximation as:

$$\nabla_\theta \widehat{\mathcal{L}} = \mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}(\widehat{Y})\right].$$

The Monte Carlo estimator, $\nabla_\theta \widehat{\mathcal{L}}_{\text{MC}}$, is obtained by calculating the gradient for each sample, $\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}(\widehat{Y}^{(i)})$. The true gradient is thus approximated by averaging the pathwise gradients from each realization:

$$\nabla_\theta \mathcal{L} \approx \underbrace{\nabla_\theta \widehat{\mathcal{L}}}_{\text{Discretization}} \approx \underbrace{\nabla_\theta \widehat{\mathcal{L}}_{\text{MC}}}_{\text{MC Sampling}} = \frac{1}{M}\sum_{i=1}^{M}\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}(\widehat{Y})^{(i)}$$

3.1.1 Accuracy and Training Stability of Gradient Estimation

The MC gradient, by its definition, functions as an unbiased estimator for the true gradient. The accuracy of this gradient is determined by both the number of time steps used in the discretization of the SDEs and the number of samples. Increasing the resolution of time steps and the sample size leads to a more accurate gradient estimation.

Assume we want to estimate the gradient value $\nabla_\theta \mathcal{L} = \mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]$ with an accuracy ϵ. We can write the mean square error(MSE) of the MC gradient estimation as in equation 6 [Burgos, 2014].

$$\mathbb{E}\left[(\nabla_\theta \widehat{\mathcal{L}}_{\text{MC}} - \nabla_\theta \mathcal{L})^2\right] = \frac{1}{M}\mathbb{V}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}(\widehat{Y})\right] + \left(\mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}(\widehat{Y})\right] - \mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]\right)^2 \tag{6}$$

The first term represents the variance of the estimator due to Monte Carlo sampling and the second term represents the bias due to the discretization. With any solver with weak order of convergence of 1, such as Milstein method [Milstein, 1994], we need $O(\epsilon^{-1})$ time steps and $O(\epsilon^{-2})$ samples to achieve MSE $O(\epsilon^2)$. Therefore, the total computational cost is $O(\epsilon^{-3})$[Duffie and Glynn, 1995].

The stability of model training is closely related to the norm of the gradient, as demonstrated in previous studies Chen et al. [2018b], Zhang et al. [2019]. The MC integration method offers an advantage in reducing the norm of the gradients compared to its non-MC counterpart, particularly during the early stages of training when the inherent stochasticity often leads to exploding gradients. The MC estimator, with a lower variance than the non-MC estimator when the number of samples, M, is sufficiently large, contributes to training stability. This stability is further demonstrated in the subsequent experimental section.

4 Multilevel Monte Carlo-Neural SDEs

While MC integration enhances gradient estimation accuracy and training stability, it is not scalable due to its $O(\epsilon^{-3})$ computational cost to achieve an MSE of $O(\epsilon^2)$. Therefore, this section introduces the MLMC method as a solution to enhance the computational efficiency of gradient estimation.

4.1 Multilevel Monte Carlo Integration for Gradient Estimation in Neural SDEs

The MLMC approach significantly improves the computational efficiency of gradient estimation by leveraging a hierarchy of approximations at different levels of granularity. This is achieved by approximating the gradient of the loss function, $\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_L(Y)\right]$, as shown in equation 7:

$$\begin{aligned}\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}_L(Y)\right] &\approx \mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_L(\widehat{Y})\right] \\ &= \mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_0(\widehat{Y})\right] + \sum_{\ell=1}^{L}\mathbb{E}_W\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_\ell(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_{\ell-1}(\widehat{Y})\right].\end{aligned} \tag{7}$$

In this approximation, the gradient is calculated as the sum of a base approximation at level 0 and a series of corrective terms from level 1 to level L.

To further facilitate the computation, for each level $l = 0, \ldots, L$, we establish independent multilevel estimators, $\widehat{U}_l$, each with a computational complexity of C_l and based on M_l samples, as defined in equation equation 8:

$$\begin{aligned}\widehat{U}_0 &= M_0^{-1}\sum_{i=1}^{M_0}\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_0(\widehat{Y})^{(i)} \\ \widehat{U}_l &= M_l^{-1}\sum_{i=1}^{M_l}\left(\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y})^{(i)} - \frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_{l-1}(\widehat{Y})^{(i)}\right).\end{aligned} \tag{8}$$

The $\widehat{U}_0$ serves as an unbiased estimator for the expectation of the gradients at the coarsest level. For each $l = 1, \ldots, L$, $\widehat{U}_l$ is an unbiased estimator of the expectation of the difference between the gradients calculated at two consecutive levels of resolution, l and $l-1$.

The MLMC estimator of the true gradient, denoted as $\nabla_\theta\widehat{\mathcal{L}}_{\text{MLMC}}$, is then defined as the sum of the coarsest level estimator and the corrections from all levels: $\widehat{U}_0 + \sum_{l=1}^{L}\widehat{U}_l$. The complete process of computing this MLMC estimator is delineated in Algorithm 1.

Algorithm 1 MLMC estimation of $\nabla_\theta\mathcal{L}$

```
Initialize ∇_θ L̂_MLMC ← 0
for l = 0 to L do
    Initialize Û_l ← 0
    for i = 1 to M_l do
        if l = 0 then
            Û_0^(i) ← (d/dθ) L̂_0(Ŷ)^(i)
        else
            Generate a coarse path with N_c(l) = 2^(l−1) time steps
            Generate a fine path with N_f(l) = 2^l time steps
            Calculate Û_l^(i) ← (d/dθ) L̂_l(Ŷ)^(i) − (d/dθ) L̂_{l−1}(Ŷ)^(i)
        end if
        Update Û_l ← Û_l + (1/M_l) Û_l^(i)
    end for
    Update ∇_θ L̂_MLMC ← ∇_θ L̂_MLMC + Û_l
end for
    return ∇_θ L̂_MLMC
```

In the Algorithm 1, we use subscripts f and c to indicate values associated with the fine and coarse levels, respectively, at a particular level l of the simulation. For the sake of computational efficiency, we calculate $\widehat{U}_l$ as $M_l^{-1}\sum_{i=1}^{M_l}\left(\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_f - \frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_c\right)$.

In this context, fine discretizations, denoted as $\widehat{\mathcal{L}}_f$, are computed using $N_f(l) = 2^l$ fine time steps, whereas coarse discretizations, $\widehat{\mathcal{L}}_c$, are computed using $N_c(l) = 2^{l-1}$ as indicated by Burgos [2014].

It's important to clarify that both fine and coarse levels are essentially two different discretizations of the same underlying Brownian process. This approach is advantageous as it reduces computational costs by obviating the need to generate two distinct Brownian processes. Instead, we can derive the increments of the coarse level's Brownian process, ΔW_c, by summing the increments of the fine Brownian process, ΔW_f, in pairs.

4.2 Computational Complexity of MLMC-Neural SDEs

Let us denote $V_l = M_l\mathbb{V}[\widehat{U}_l]$ as the variance of an individual sample from the multilevel estimator (i.e., $\mathbb{V}[\widehat{U}_l^{(i)}]$). Based on the settings outlined in the previous subsection, we can express the computational cost and variance of the MLMC estimator, $\nabla_\theta\widehat{\mathcal{L}}_{\text{MLMC}}$, as proportional to $\sum_{l=0}^{L} M_l 2^l$ and $\sum_{l=0}^{L} V_l M_l^{-1}$ respectively [Ishikawa and Goda, 2021].

To ensure the variance of the MLMC estimator, $\nabla_\theta\widehat{\mathcal{L}}_{\text{MLMC}}$, is appropriately constrained as per:

$$\sum_{l=0}^{L}\frac{V_l}{M_l} \leq \frac{\epsilon^2}{2}.$$

we operate under the assumption that the variance V_l decays following $2^{-\beta l}$ for increasing l, where $\beta > 0$. This assumption motivates the strategy to exponentially decrease the sample sizes, M_l, at each level. The optimal sample sizes, $M_0, \ldots, M_L$, are thus determined by minimizing the total computational cost while ensuring the variance remains bounded by $\epsilon^2/2$:

$$\sum_{l=0}^{L} M_l 2^l + \lambda\left(\sum_{l=0}^{L}\frac{V_l}{M_l} - \frac{\epsilon^2}{2}\right). \tag{9}$$

From this, we infer that $M_l \propto \sqrt{V_l 2^{-l}}$. According to the Complexity Theorem 2.1, computational complexity is guaranteed if we can estimate the order of convergence α and β while using $M_l \propto \sqrt{V_l 2^{-l}}$ simulations for each level. We can estimate these values through simulations, offering a practical understanding of these parameters. Assuming we have,

$$\mathbb{E}\left[\widehat{U}_l\right] = O(h_l^\alpha), V_l = O(h_l^\beta),$$

where $h_l = 2^{-l}T$. Then we can estimate the empirical bounds, which approximate α and β with

$$\log_2\mathbb{E}\left[\widehat{U}_l\right] \sim \alpha\log_2(h) \sim -l\alpha$$
$$\log_2 V_l \sim \beta\log_2(h) \sim -l\beta.$$

For each level, denoted by l, where l ranges from 0 to L, we calculate the values of $\log_2\mathbb{E}[\widehat{U}_l]$ and $\log_2 V_l$ using M_l simulations. Following this, we proceed to estimate the coefficients α and β by determining the final slopes of the graphs generated through these simulations. The numerical scheme chosen to solve the SDEs typically influences the value of α, and we have found that we can usually achieve an α value close to 1.0 when we use the Milstein scheme in our experiments.

Meanwhile, the value of β is largely dependent on the overall system design (for instance, the choice of decoder following the SDE integration) and the loss function. However, even in cases where $\beta < 1$, the cost, given as $c_4\epsilon^{-2-(1-\beta)/\alpha}$, remains competitive when compared to the cost of the Monte Carlo estimator, which is at $O(\epsilon^{-3})$.

5 Related Work

Neural Differential Equations. The introduction of Neural Ordinary Differential Equations by Chen et al. [2018a] has propelled the ability to model complex dynamics by defining the vector field of a differential equation with neural networks. This approach has naturally extended to various types of differential equations, including Neural Control Differential Equations [Kidger et al., 2020] and Neural Stochastic Differential Equations [Liu et al., 2019]. Building upon this, Rubanova et al. [2019] introduced a family of time-series models, termed ODE-RNNs, where the dynamics of the RNN's hidden state are dictated by Neural ODEs. Cameron et al. [2021] presents an approach to learning SDEs by proposing an importance-sampling estimator, which bypasses the computational challenges of sequential SDE integrators.

MLMC for Gradient Estimation. Mohamed et al. [2020] provides a comprehensive study on MC methods for gradient estimation in machine learning, addressing their efficacy and applications in various learning scenarios. To improve computational complexity in methods requiring MC for gradient computations, various works have focused on optimization. Fujisawa and Sato [2021] put forward a framework for variance reduction in variational inference, employing MLMC techniques, which resulted in expedited convergence and diminished variance in gradient estimations compared to standard approaches. Shi and Cornish [2021] presents another application of MLMC methods , achieving unbiased gradient approximation during the training of deep latent variable models, exceeding the performance of existing debiasing methods at an equivalent computational cost. Goda et al. [2022] and Ishikawa and Goda [2021] incorporated MLMC into Bayesian contexts to provide unbiased evidence and gradient estimations.

6 Experiments

In this section, our focus is threefold. First, we compare the gradients produced by MC-Neural SDEs with the precise gradients derived from a problem possessing an analytical solution. This comparison enables us to empirically assess gradient accuracy and examine the balance between the number of simulation samples and gradient precision. Additionally, we analytically illustrate that MLMC-Neural SDEs, within the same architectural context, can decrease the computational complexity to $O(\epsilon^{-2})$.

Our second goal, using synthetic data, is to show how accurate gradient calculations can lead to stable training and improved convergence. We use Latent SDE as a baseline model for comparison. Furthermore, we emphasize that the MLMC method achieves performance comparable to MC, but with less computational cost.

Lastly, we apply our approach to a real-world dataset, the mocap dataset, and compare its performance with existing methods, especially when integrated with Latent SDEs. This step helps us demonstrate how well our approach performs in practical scenarios.

MC simulations in each experiment can be batch-parallelized, increasing GPU memory usage proportional to the number of samples without impacting computational time per iteration. Due to space limitations, further details on the datasets, the chosen architecture for Neural SDEs, and training specifics for each experiment are provided in the supplementary material.

6.1 Numerical Experiment

Let's assume our Neural SDEs are based on the Geometric Brownian Motion formulation:

$$\mathrm{d}X_t = \mu X_t \mathrm{d}t + \sigma X_t \mathrm{d}W_t, \quad X_0 = x_0.$$

We aim to estimate the parameters μ and σ using the loss between the log returns, $r_t = \ln(\frac{X_t}{X_{t-1}})$, of the ground-truth and our model. We employ the mean squared error as our loss function and compute the loss between the true log returns, r_t, and the predicted log returns, $\widehat{r}_t$. The predicted log returns can be rewritten as $\widehat{r}_t = (\mu - \frac{\sigma^2}{2})\Delta t$, since $X_t = X_0 \exp((\mu - \frac{\sigma^2}{2})t + \sigma W_t)$. Given the log returns, the loss between the predicted and actual log returns is defined as $\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T}(r_t - \widehat{r}_t)^2$.

Integrating MC into Neural SDEs, we calculate the approximated gradient as $\nabla_\theta \widehat{\mathcal{L}}_{\text{MC}}$. Applying 128 discretization steps and true parameter values of $\mu = 0.2, \sigma = 0.1$, we compute the error between the analytically derived true gradient, $\nabla_\theta \mathcal{L}$, and our approximated gradients. We conduct

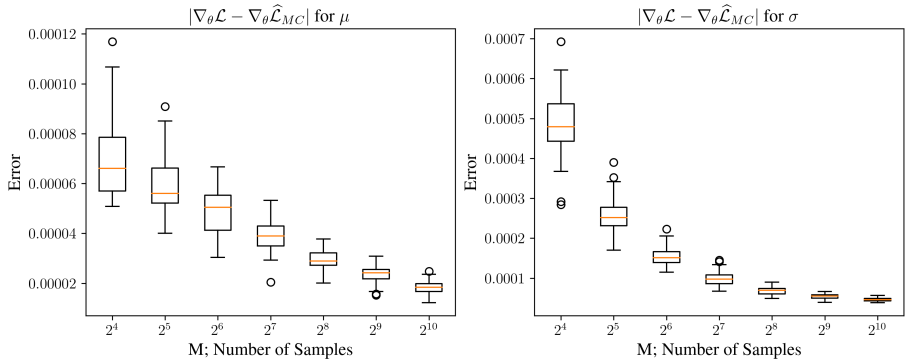

Figure 2: Error between true and approximated gradients for different numbers of MC samples

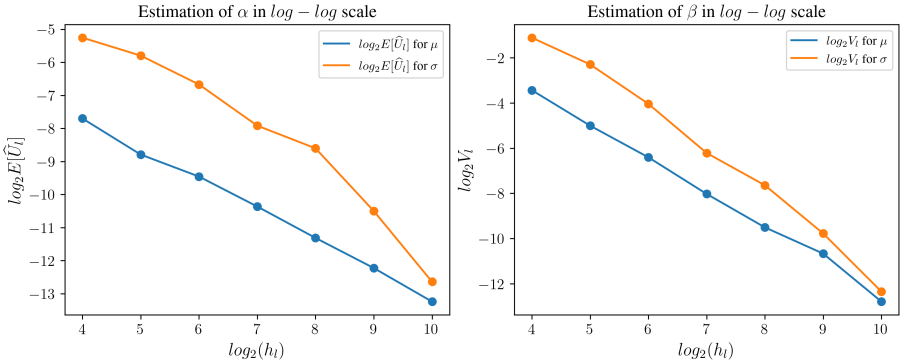

Figure 3: Estimation of convergence order for the gradients of μ and σ in MLMC-Neural SDEs

100 independent experiments for each case with different numbers of samples (M) used in MC, ranging from 2^4 to 2^{10}, as shown in Figure 2. The results illustrate that the MC-Neural SDEs can accurately compute the gradient, and the gradient's accuracy and training robustness (variance of the gradients) can be adjusted by trading off the number of samples M used for MC simulation.

Next, we assess the computational cost of MLMC-Neural SDEs for comparable gradient accuracy to MC-Neural SDEs. By applying Complexity Theorem 2.1 and estimating convergence order α and β using equation equation 9, we find that for our Neural SDEs architecture and loss function, $\alpha \approx 0.9$ and $\beta \approx 1.51$ for the μ gradient, and $\alpha \approx 1.1$ and $\beta \approx 1.86$ for the σ gradient. Given $\beta > 1$ for both parameters, MLMC-Neural SDEs can indeed achieve computational complexity of $O(\epsilon^{-2})$, indicating effective cost reduction.

6.2 Synthetic Datasets

Using the 3D Stochastic Lorenz Attractor synthetic dataset, we investigate the balance between loss and computational cost. This experiment also demonstrates the reduction of the gradient norm through MC simulation. We employ Latent SDEs as a choice of our Neural SDEs, in line with the experimental setup of Li et al. [2020], to integrate MC and MLMC methodologies. Given its superior performance in various generative tasks such as multi-dimensional synthetic and motion capture data, Latent SDEs are utilized in this and the following sections.

The computational cost for Base LatentSDE is proportional to the number of time steps, N, whereas, for MC-LatentSDE, it is proportional to the product of the number of time steps and the number of samples, NM. For MLMC-LatentSDE, the cost is proportional to $\sum_{l=0}^{L} N_f(l) M_l$. We compute the average loss value across 20 independent trainings for all models, training each until convergence. As depicted in Figure 6.2 (left), the precise gradient of MC and MLMC methods yield superior convergence loss compared to Base LatentSDE. Remarkably, MLMC-LatentSDE achieves comparable performance to MC-LatentSDE with over 50% less computational cost. The Figure 6.2(right)

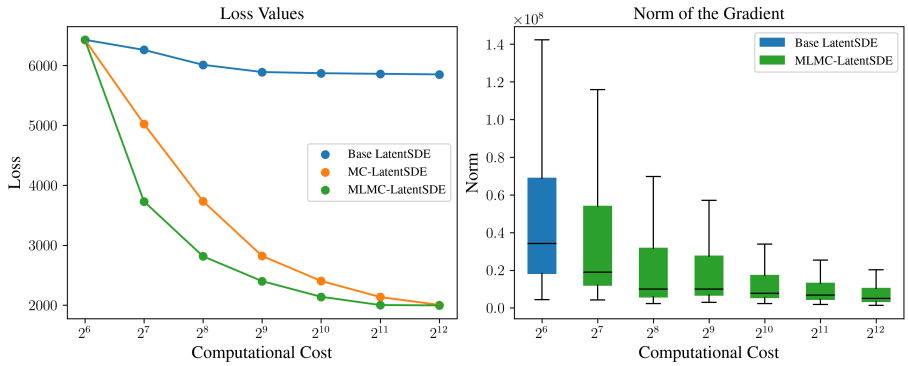

Figure 4: Performance comparison of Base LatentSDE, MC-LatentSDE, and MLMC-LatentSDE

illustrates the gradient norm reduction for MLMC-LatentSDE, indicating that lower gradient norms contribute to stable training and rapid convergence in our experiment. This suggests an advantageous trade-off between computational cost and gradient norm.

6.3 Motion Capture Datasets

The significance of precise gradient computation is further highlighted using real-world datasets. We deploy a 50-dimensional motion capture dataset from Gan et al. [2015] to evaluate our model's predictive accuracy. The task involves predicting 297 future frames after observing the initial three frames from a subject's walking sequences.

The table 1 reports the test Mean Squared Error (MSE) on future frames. By incorporating MLMC, Latent SDE demonstrates an improvement in test MSE. While naive MC integration can achieve comparable performance, it does so at a higher computational cost. In our experiment, MLMC-LatentSDE achieves the same performance as MC integration while reducing the computational cost by over 30%.

Table 1: Average test MSE on 297 future frames with 95% confidence interval under t-statistic. †results from Yildiz et al. [2019] and ‡ results from Li et al. [2020].

Method	Test MSE	Reference
NeuralODE	$22.49 \pm 0.88^{\dagger}$	Chen et al. [2018a]
ODE^{2}VAE-KL	$8.09 \pm 1.95^{\dagger}$	Yildiz et al. [2019]
Latent ODE	$5.98 \pm 0.28^{\ddagger}$	Rubanova et al. [2019]
Latent SDE	$4.03 \pm 0.20^{\ddagger}$	Li et al. [2020]
MLMC-Latent SDE	$\mathbf{3.85 \pm 0.17}$	current work

7 Discussion

Despite the advancements presented in our work, certain avenues remain open for further exploration. One inherent limitation of Neural SDEs employing discretization solvers pertains to the error introduced by the discretization process. This error can only be mitigated by reducing the step size. Addressing this issue, and subsequently applying MC or MLMC methods presents a promising direction for future research.

Despite the prospects for future work, this study presents a novel approach to integrating MLMC into Neural SDE training. Our proposed method generates gradient estimates that are both accurate and computationally efficient. This precision in gradient calculation enhances the base Neural SDE model's performance, as evidenced by improved loss convergence and stability across our numerical, synthetic, and real-world experiments. The results underscore the potential and efficacy of our approach, and we believe it serves as a foundation for further explorations in this field.

References

F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of political economy*, 81(3):637–654, 1973.

L. Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010.

S. Burgos. *The computation of Greeks with multilevel Monte Carlo*. PhD thesis, University of Oxford, 2014.

S. Cameron, T. Cameron, A. Pretorius, and S. Roberts. Robust and scalable sde learning: A functional perspective. *arXiv preprint arXiv:2110.05167*, 2021.

R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018a.

Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *International conference on machine learning*, pages 794–803. PMLR, 2018b.

K. A. Cliffe, M. B. Giles, R. Scheichl, and A. L. Teckentrup. Multilevel monte carlo methods and applications to elliptic pdes with random coefficients. *Computing and Visualization in Science*, 14 (1):3–15, 2011.

R. Deng, M. A. Brubaker, G. Mori, and A. Lehrmann. Continuous latent process flows. *Advances in Neural Information Processing Systems*, 34:5162–5173, 2021.

D. Duffie and P. Glynn. Efficient monte carlo simulation of security prices. *The Annals of Applied Probability*, pages 897–905, 1995.

M. Fujisawa and I. Sato. Multilevel monte carlo variational inference. *The Journal of Machine Learning Research*, 22(1):12741–12784, 2021.

Z. Gan, C. Li, R. Henao, D. E. Carlson, and L. Carin. Deep temporal sigmoid belief networks for sequence modeling. *Advances in Neural Information Processing Systems*, 28, 2015.

C. W. Gardiner et al. *Handbook of stochastic methods*, volume 3. springer Berlin, 1985.

M. Giles. Improved multilevel monte carlo convergence using the milstein scheme. In *Monte Carlo and Quasi-Monte Carlo Methods 2006*, pages 343–358. Springer, 2008a.

M. Giles and P. Glasserman. Smoking adjoints: Fast monte carlo greeks. *Risk*, 19(1):88–92, 2006.

M. B. Giles. Multilevel monte carlo path simulation. *Operations research*, 56(3):607–617, 2008b.

M. B. Giles. Multilevel monte carlo methods. *Acta numerica*, 24:259–328, 2015.

E. Gobet and R. Munos. Sensitivity analysis using itô–malliavin calculus and martingales, and application to stochastic optimal control. *SIAM Journal on control and optimization*, 43(5): 1676–1713, 2005.

T. Goda, T. Hironaka, W. Kitade, and A. Foster. Unbiased mlmc stochastic gradient-based optimization of bayesian experimental designs. *SIAM Journal on Scientific Computing*, 44(1):A286–A311, 2022.

J. Hull and A. White. Pricing interest-rate-derivative securities. *The review of financial studies*, 3(4): 573–592, 1990.

K. Ishikawa and T. Goda. Efficient debiased evidence estimation by multilevel monte carlo sampling. In *Uncertainty in Artificial Intelligence*, pages 34–43. PMLR, 2021.

P. Kidger. On neural differential equations. *arXiv preprint arXiv:2202.02435*, 2022.

P. Kidger, J. Morrill, J. Foster, and T. Lyons. Neural controlled differential equations for irregular time series. *Advances in Neural Information Processing Systems*, 33:6696–6707, 2020.

P. Kidger, J. Foster, X. Li, and T. J. Lyons. Neural sdes as infinite-dimensional gans. In *International Conference on Machine Learning*, pages 5453–5463. PMLR, 2021a.

P. Kidger, J. Foster, X. C. Li, and T. Lyons. Efficient and accurate gradients for neural sdes. *Advances in Neural Information Processing Systems*, 34:18747–18761, 2021b.

X. Li, T.-K. L. Wong, R. T. Chen, and D. Duvenaud. Scalable gradients for stochastic differential equations. In *International Conference on Artificial Intelligence and Statistics*, pages 3870–3882. PMLR, 2020.

X. Liu, T. Xiao, S. Si, Q. Cao, S. Kumar, and C.-J. Hsieh. Neural sde: Stabilizing neural ode networks with stochastic noise. *arXiv preprint arXiv:1906.02355*, 2019.

G. N. Milstein. *Numerical integration of stochastic differential equations*, volume 313. Springer Science & Business Media, 1994.

S. Mohamed, M. Rosca, M. Figurnov, and A. Mnih. Monte carlo gradient estimation in machine learning. *The Journal of Machine Learning Research*, 21(1):5183–5244, 2020.

B. Øksendal and B. Øksendal. *Stochastic differential equations*. Springer, 2003.

Y. Rubanova, R. T. Q. Chen, and D. Duvenaud. Latent odes for irregularly-sampled time series, 2019.

Y. Shi and R. Cornish. On multilevel monte carlo unbiased gradient estimation for deep latent variable models. In *International Conference on Artificial Intelligence and Statistics*, pages 3925–3933. PMLR, 2021.

B. Tzen and M. Raginsky. Neural stochastic differential equations: Deep latent gaussian models in the diffusion limit. *arXiv preprint arXiv:1905.09883*, 2019.

C. Yildiz, M. Heinonen, and H. Lahdesmaki. Ode2vae: Deep generative second order odes with bayesian neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.

J. Zhang, T. He, S. Sra, and A. Jadbabaie. Why gradient clipping accelerates training: A theoretical justification for adaptivity. *arXiv preprint arXiv:1905.11881*, 2019.

A Details on MLMC Complexity Theorem

The structure of Theorem 2.1 is based on the work presented in Burgos [2014]. The original proof for the Multi-Level Monte Carlo (MLMC) Complexity Theorem can be found in Giles [2008a], with an extended version available in Cliffe et al. [2011]. Here, we provide a simplified overview of the proof, guided by the method outlined in Ishikawa and Goda [2021].

We'll define $V_l = M_l \mathbb{V}[\widehat{U}_l]$ and $\mathcal{C}_l = M_l^{-1} C_l$ to represent the variance and the cost per sample from the multi-level estimator, respectively. With these definitions, the computational cost and variance of the MLMC estimator, denoted as $\nabla_\theta \widehat{\mathcal{L}}_{\text{MLMC}}$, can be expressed as follows:

$$\sum_{l=0}^{L} \mathcal{C}_l M_l \quad \text{and} \quad \sum_{l=0}^{L} V_l M_l^{-1},$$

respectively. We'll consider $h_l = 2^{-l}$ and assume the existence of positive constants $\alpha \geq \frac{1}{2}\min(1, \beta)$ β, c_1, c_2, c_3, that satisfy the following conditions:

$$\begin{aligned}
&A1 : \mathbb{E}\left[\widehat{U}_l\right] = \begin{cases} \mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_0(\widehat{Y})\right] & \text{if } l = 0 \\ \mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_{l-1}(\widehat{Y})\right] & \text{if } l > 0 \end{cases} \\
&A2 : \left|\mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_l(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]\right| \leq c_1 2^{-\alpha l} \\
&A3 : V_l \leq c_2 2^{-\beta l} \\
&A4 : \mathcal{C}_l \leq c_3 2^{l}.
\end{aligned} \tag{10}$$

The mean squared error (MSE) of the MLMC estimator can be broken down into the sum of the variance and the squared bias. In order to ensure this sum is not greater than ϵ^2, it is sufficient for the following conditions to be met:

$$\left|\mathbb{E}\left[\frac{\mathrm{d}}{\mathrm{d}\theta}\widehat{\mathcal{L}}_L(\widehat{Y}) - \frac{\mathrm{d}}{\mathrm{d}\theta}\mathcal{L}(Y)\right]\right|^2 \leq \frac{\epsilon^2}{2} \quad \text{and} \quad \sum_{l=0}^{L} \frac{V_l}{M_l} \leq \frac{\epsilon^2}{2}.$$

By using the second condition in equation 10, we can select the maximum level, L, as follows:

$$L = \left\lceil \frac{\log_2\left(\sqrt{2} c_1 \epsilon^{-1}\right)}{\alpha} \right\rceil,$$

where L satisfies $c_1 2^{-\alpha l} \leq \epsilon/\sqrt{2}$. The optimal sample sizes, $M_0, \ldots, M_L$ are then determined by minimizing the total computational cost while ensuring the variance remains below $\epsilon^2/2$:

$$\sum_{l=0}^{L} \mathcal{C}_l M_l + \lambda \left(\sum_{l=0}^{L} \frac{V_l}{M_l} - \frac{\epsilon^2}{2} \right). \tag{11}$$

For each level l, the optimal value of M_l can be computed as follows:

$$M_l = \sqrt{\frac{\lambda V_l}{\mathcal{C}_l}}.$$

By differentiating equation 11 with respect to λ and substituting M_l with the calculated value, we derive the following:

$$\sum_{l'=0}^{L} V_{l'} \sqrt{\frac{\mathcal{C}_{l'}}{\lambda V_{l'}}} = \frac{\epsilon^2}{2}$$

and by substitution,

$$\sqrt{\lambda} = 2\epsilon^{-2} \sum_{l'=0}^{L} \sqrt{\mathcal{C}_{l'} V_{l'}}$$

This allows us to implicitly define λ as a function of the levels l. The optimal value of M_l can be reformulated through straightforward calculation as

$$M_l = \left\lceil 2\epsilon^{-2}\sqrt{\frac{V_l}{\mathcal{C}_l}}\sum_{l'=0}^{L}\sqrt{\mathcal{C}_{l'}V_{l'}}\right\rceil.$$

Next, we compute the upper bound of the computational cost:

$$\begin{aligned}\sum_{l=0}^{L}\mathcal{C}_l M_l &\leq \sum_{l=0}^{L}\mathcal{C}_l\left(1+2\epsilon^{-2}\sqrt{\frac{V_l}{\mathcal{C}_l}}\sum_{l'=0}^{L}\sqrt{\mathcal{C}_{l'}V_{l'}}\right)\\ &= \sum_{l=0}^{L}\mathcal{C}_l + 2\epsilon^{-2}\left(\sum_{l=0}^{L}\sqrt{\mathcal{C}_l V_l}\right)^2.\end{aligned}$$

By using conditions 3 and 4 from equation 10, we can derive the following bound:

$$\sum_{l=0}^{L}\mathcal{C}_l M_l \leq c_3\sum_{l=0}^{L}2^l + 2c_2c_3\epsilon^{-2}\left(\sum_{l=0}^{L}2^{-\beta l/2}\right)^2$$

The proof can be efficiently concluded via direct calculations, where each of the three distinct cases for β ($\beta > 1$, $\beta = 1$, and $0 < \beta < 1$) contributes uniquely to the complexity of the MLMC estimator.

B Details on Experiments

This section presents specifics related to the training process, including details of the dataset and the selected architecture for Neural SDEs. All experiments were performed on a single GPU, specifically the NVIDIA GeForce RTX 3090. The primary implementation is anonymously available at the following URL: https://anonymous.4open.science/r/MLMCNeuralSDEs-7EEC/.

B.1 Numerical Experiments

The form of Neural SDEs selected for our numerical experiments is as follows:

$$\mathrm{d}X_t = \mu X_t \mathrm{d}t + \sigma X_t \mathrm{d}W_t, \quad X_0 = x_0,$$

Here, x_0 represents the initial value for the SDE. Considering that $X_t = X_0\exp((\mu - \frac{\sigma^2}{2})t + \sigma W_t)$, we define the true log returns, r_t, and the predicted log returns, $\widehat{r}_t$, as:

$$r_t = \ln(\frac{X_t}{X_{t-1}}) \quad \text{and} \quad \widehat{r}_t = (\mu - \frac{\sigma^2}{2})\Delta t.$$

The mean squared error is selected as the loss function, and the expression for the loss value is:

$$\textit{MSE}: \mathcal{L} = \frac{1}{T}\sum_{t=1}^{T}(r_t - \widehat{r}_t)^2 = \frac{1}{T}\sum_{t=1}^{T}(r_t - (\mu - \frac{\sigma^2}{2})\Delta t).$$

The analytical derivatives of each parameter with respect to the loss can be computed as:

$$\begin{aligned}\nabla_\mu\mathcal{L} &= \frac{\partial\mathcal{L}}{\partial\mu} = \frac{1}{T}\sum_{t=1}^{T}-2(r_t - (\mu - \frac{\sigma^2}{2})\Delta t)\Delta t\\ \nabla_\sigma\mathcal{L} &= \frac{\partial\mathcal{L}}{\partial\sigma} = \frac{1}{T}\sum_{t=1}^{T}-2(r_t - (\mu - \frac{\sigma^2}{2})\Delta t)(-\sigma\Delta t).\end{aligned}$$

B.2 Latent SDEs

Following the experimental setup by Li et al. [2020], we choose Latent SDEs as our Neural SDEs for the experiments with synthetic and motion capture datasets. We provide details of Latent SDEs here.

Consider a series of observations $\{(t_i, x_{t_i})\}$ within a finite time frame $\mathbb{T} = [0, T]$, where x_{t_i} is m-dimensional. A filtered probability space is denoted by $(\Omega, \mathcal{F}_t, P)$. We intend to determine an approximate posterior process regulated by the following SDE:

$$\mathrm{d}Z_t = f(t, Z_t)\mathrm{d}t + g(t, Z_t)\mathrm{d}W_t.$$

Here, W_t represents a d-dimensional Brownian process, and the initial condition of the process Z_0 is $z_0 \in \mathbb{R}^d$. Assume there is a measurable function $u(t, z)$ fulfilling $g(t, z)u(t, z) = f(t, z) - h(t, z)$, and that $u(t, Z_t)$ adheres to Novikov's condition:

$$\mathbb{E}\left[\exp\left(\int_0^T \frac{1}{2}\left|u\left(t, Z_t\right)\right|^2 \mathrm{d}t\right)\right] < \infty.$$

Applying Girsanov's Theorem, we can establish a measurable space $(\Omega, \mathcal{F}_t)$, paired with a probability measure Q, such that the process $\widehat{W}_t = \int_0^t u(s, Z_s)\mathrm{d}s + W_t$ defines another Brownian process. Let's denote the importance weight as M_t. The satisfaction of Novikov's condition guarantees that the process

$$M_t = \exp\left(-\int_0^t \frac{1}{2}\left|u\left(s, Z_s\right)\right|^2 \mathrm{d}s - \int_0^t u\left(s, Z_s\right)^\top \mathrm{d}W_s\right)$$

is a P-martingale and the probability measure Q is defined by $\mathrm{d}Q = M_T \mathrm{d}P$. Then we can show that

$$\mathrm{d}Z_t = h(t, Z_t)\mathrm{d}t + g(t, Z_t)\mathrm{d}\widehat{W}_t.$$

We can therefore deduce that the Q-law of the approximate posterior is the same as the P-law of the prior. Therefore when calculating the variational bound, instead of sampling from the unknown distribution under Q, we can estimate with known distribution under P by applying the importance weight M_t.

$$\mathbb{E}_P\left[\prod_{i=1}^N p\left(x_{t_i}|\tilde{z}_{t_i}\right)\right] = \mathbb{E}_Q\left[\prod_{i=1}^N p\left(x_{t_i}|z_{t_i}\right)\right] = \mathbb{E}_P\left[\prod_{i=1}^N p\left(x_{t_i}|z_{t_i}\right) M_T\right],$$

where the prior process follows $\mathrm{d}\tilde{Z}_t = h(t, \tilde{Z}_t)\mathrm{d}t + g(t, \tilde{Z}_t)\mathrm{d}W_t$. Then, by simple calculation, we can get the evidence lower bound (ELBO) formula for observing $x_1, x_2, \ldots, x_N$ is given as:

$$\log p(x_{t_1}, x_{t_2}, ..., x_{t_N}) \geq \mathbb{E}_P\left[\sum_{i=1}^N \log p\left(x_{t_i}|z_{t_i}\right) - \int_0^T \frac{1}{2}\left|u\left(t, Z_t\right)\right|^2 \mathrm{d}t\right]. \tag{12}$$

Algorithm 2 illustrates a pseudocode implementation of Latent SDEs. In the context of training, the model seeks to maximize the evidence lower bound, achieved by evaluating the right-hand side of the equation 12. This is a high-level explanation of the procedure. A detailed understanding of the specific implementation can be done with the code provided.

Algorithm 2 Pseudocode of Latent SDEs

Input: Observations $\{(t_i, x_{t_i})\}$, for $i = 1, \ldots, N$
if Contextualize **then**
  $\{C_{t_i}\}_{i=1}^N$ = *Encoder*$(\{x_{t_i}\}_{i=1}^N)$ ▷ Save context variables for posterior drift
  μ_{z_0}, σ_{z_0} = *Linear*(C_{t_1})
else
  μ_{z_0}, σ_{z_0} = *Encoder*$(\{x_{t_i}\}_{i=1}^N)$
end if
$z_{t_0} \sim \mathcal{N}(\mu_{z_{t_0}}, \sigma_{z_{t_0}})$ ▷ Sample initial value for the latent state
$\{z_{t_i}\}_{i=1}^N, D_{kl}$ = *NumericalSolver*$(f, h, g, z_{t_0}, (t_1 \ldots t_N))$ ▷ $D_{kl} = \int_0^T \frac{1}{2}\left|u\left(t, Z_t\right)\right|^2 \mathrm{d}t$
$\{\widehat{x}_{t_i}\}_{i=1}^N$ = *Decoder*$(\{z_i\})$, for $i = 1, \ldots, N$
$\log p(X|Z)$ = *LogProb*$(\{\widehat{x}_{t_i}\}_{i=1}^N, \{x_{t_i}\}_{i=1}^N)$ ▷ Calculate log probability for observations
Output: $\log p(X|Z), D_{kl}$

B.3 Synthetic Datasets

To assess the capability of our method in capturing various continuous dynamics, we employ synthetic data derived from a well-defined stochastic process. We specifically utilize a 3D stochastic Lorenz attractor process, chosen for its multidimensional properties, for generating our dataset.

In alignment with the settings adopted in preceding studies by Deng et al. [2021], Li et al. [2020], Kidger [2022], we employ a stochastic Lorenz attractor SDE characterized by diagonal noise:

$$\begin{aligned} y &\sim \mathcal{N}\left(0, I_{3\times 3}\right) \\ \mathrm{d}y_1(t) &= a_1\left(y_2(t) - y_1(t)\right)\mathrm{d}t + b_1 y_1(t)\mathrm{d}w(t) \\ \mathrm{d}y_2(t) &= \left(a_2 y_1(t) - y_1(t)y_3(t)\right)\mathrm{d}t + b_2 y_2(t)\mathrm{d}w(t) \\ \mathrm{d}y_3(t) &= \left(y_1(t)y_2(t) - a_3 y_3(t)\right)\mathrm{d}t + b_3 y_3(t)\mathrm{d}w(t), \end{aligned} \tag{13}$$

with $a_1 = 10$, $a_2 = 28$, $a_3 = 8/3$, $b_1 = 0.1$, $b_2 = 0.28$, $b_3 = 0.3$. Our task involves learning the underlying dynamics of the dataset using the observed data points $\{x_{t_i}\}_{i=1}^N$. We generate a total of 1024 individual time series datasets from the process, each consisting of observations at 128 distinct intervals between the time span of 0 to 2. Figure 5 visually represents the original observations and the generated sample data points $\{\widehat{x}_{t_i}\}_{i=1}^N$ subsequent to the training phase.

In our experiments, the architecture of the Latent SDEs is based on the GRU model as suggested by Li et al. [2020]. Here, a GRU is utilized as the encoder to provide contextualization, while a linear layer is deployed to compute μ_{z_0} and σ_{z_0}. The drift components for both the prior and approximate posterior are represented by Multilayer Perceptrons (MLPs) that encompass hidden layers consisting of 128 units. Similarly, the diffusion function is also defined as an MLP, but with a single hidden layer of 128 units, and includes the application of a sigmoid activation function at its end.

The datasets being studied are three-dimensional in nature, while a four-dimensional latent space is employed. Mapping from this four-dimensional latent space to the observation space is accomplished by a linear decoder. For every nonlinearity present, the softplus function is utilized.

In our experimental setup, we utilize the Adam optimizer, starting with a learning rate of 0.01, which is subsequently reduced by a factor of 0.999 with each iteration. We implement a linear KL annealing schedule over the first 1000 iterations.

Three training approaches are considered - Base LatentSDE, MC-LatentSDE, and MLMC-LatentSDE - each escalating in computational cost from 2^6 to 2^{12}. Each of these techniques slightly modifies its approach to maximize the benefits of the incremental computational cost.

In the case of Base LatentSDE, we elevate the number of steps for a fixed discretization within the SDE solver to match the computational cost that is allocated for MC-LatentSDE and MLMC-LatentSDE. For MC-LatentSDE, we increase both the number of samples and the number of steps to align the computational cost for an equitable comparison. For the MLMC-LatentSDE, we allocate

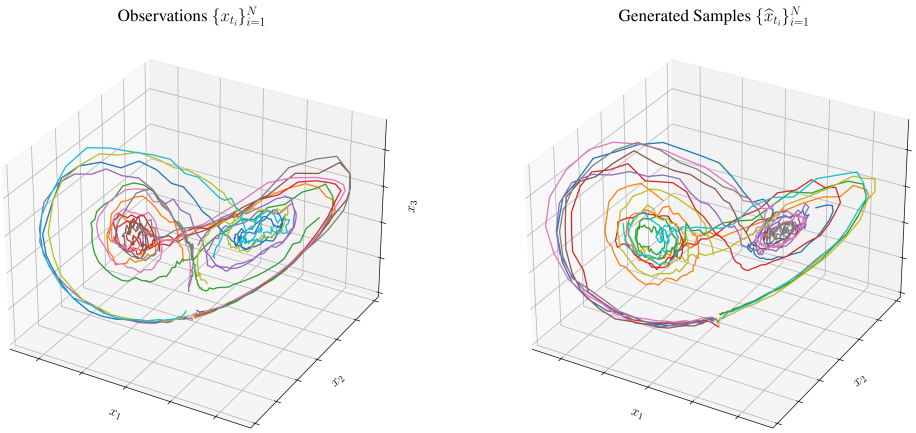

Figure 5: Observations and Generated Samples for the Stochastic Lorenz Dataset

the computational resources by adjusting the number of levels, the sample sizes for each level, and the step sizes in the discretization process.

All the methods were trained until their convergence, with a particular focus on the Base LatentSDE. For MC and MLMC-LatentSDE, less than 500 iterations were sufficient to achieve the reported loss as shown in the accompanying figure. However, for the Base LatentSDE, we needed to extend the training for over 2000 iterations until we observed that the convergence level could not be further reduced.

B.4 Motion Capture Datasets

In order to assess our method's proficiency in learning complex dynamics from real-world datasets, we employ the CMU motion capture dataset. This dataset includes 23 walking sequences pertaining to subject 35 with the length of 300, as referenced in the study by Gan et al. [2015]. The dataset is divided into 16 training sequences, 3 validation sequences, and 4 test sequences, following the partitions established in earlier literature. The data is processed in accordance with the procedure detailed by **?**, which results in 50-dimensional joint angle measurements.

Given the first three frames, the objective is to predict the future 297 frames in the test dataset. To ensure a fair comparison of results, we replicate the architecture of the Latent SDE model as detailed in Li et al. [2020]. As depicted in Figure 7, the model employs an MLP encoder that processes the first three frames, producing the mean and log-variance of the variational distribution of the initial latent state as well as a context vector. Even though the implementation of alternative encoders, such as RNN or GRU, could potentially provide enhanced results, we adhere to the structure established in prior studies for experimental consistency.

Transitioning to the decoder component, it mirrors the structure of the encoder, projecting the 6-dimensional latent state into the 50-dimensional observation space. The drift function of the posterior incorporates a 3-dimensional context vector, which is the output from the encoder, along with the current state and time. It is important to note that during our experimental setup, the inclusion of the context vector did not provide a significant improvement in performance. However, its incorporation has been retained in the models. Conversely, the prior drift solely depends on the current state and time. The diffusion function is constructed of several smaller MLPs, each producing a scalar for the corresponding dimension, thus ensuring the posterior SDE features diagonal noise.

In our experiments, we employ the Adam optimizer, initiating with a learning rate of 0.001, which undergoes exponential decay at a rate of 0.9999. We also implement a linear KL annealing schedule during the first 400 iterations of the training process. Each model is trained for a total of 1000 iterations. It is important to note that the results reported in this paper, except the MLMC-LatentSDE, originate from previously published works, and the appropriate citations have been included in the respective table.

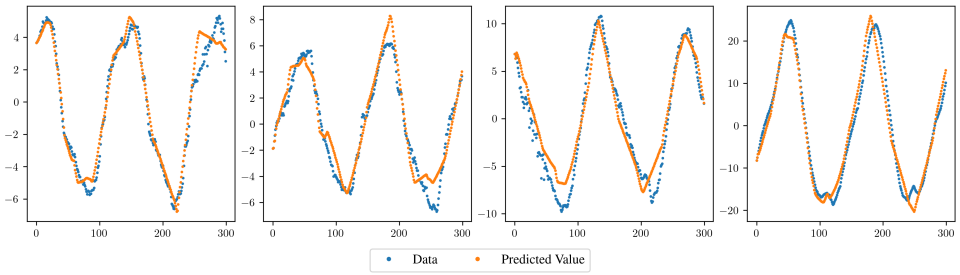

Figure 6: Comparison on the test data and predicted value with first three observations

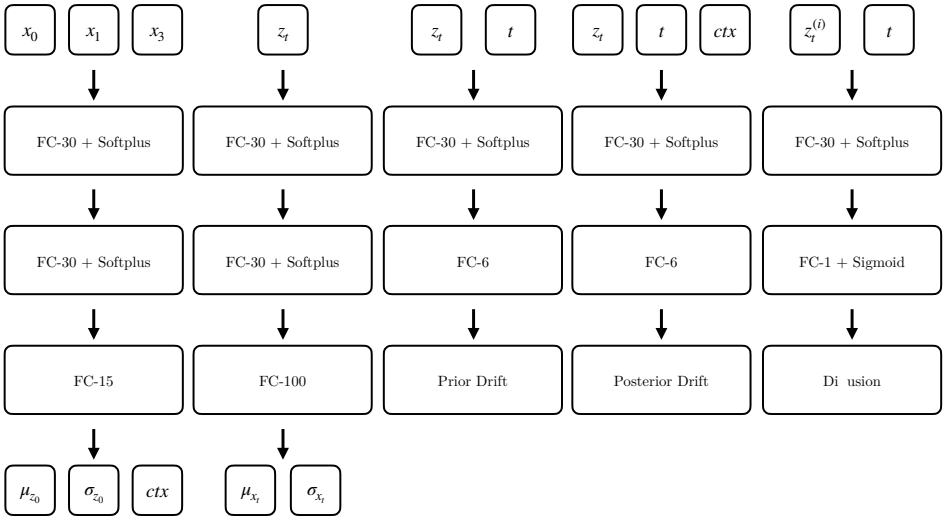

Figure 7: The architecture of Latent SDE for motion capture dataset. From left to right: encoder, decoder, prior drift, posterior drift, and diffusion functions